

《面向对象程序设计》

期末串讲

柳政尧

Contents

- Package
- Serialization
- Network Programming
- Threading
- Class+
- Functional Programming
- Design Patterns

Package

Package management in Java

包管理的作用

In Java, we use packages to **group** related classes, interfaces, and sub-packages.

包管理的好处

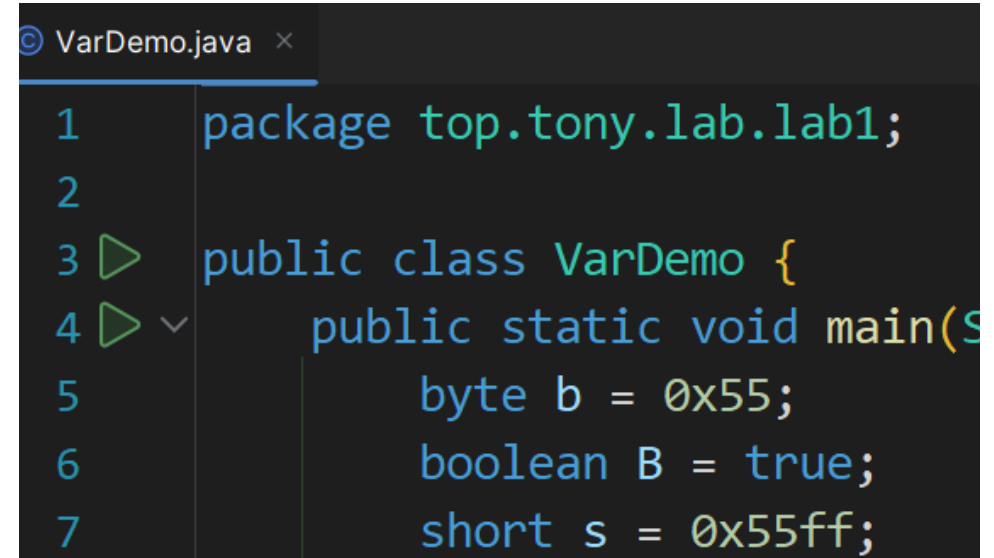
- 方便管理 —— 包之间有一定的**逻辑**相关性
- 避免命名冲突
- 权限控制

<https://www.baeldung.com/java-packages>

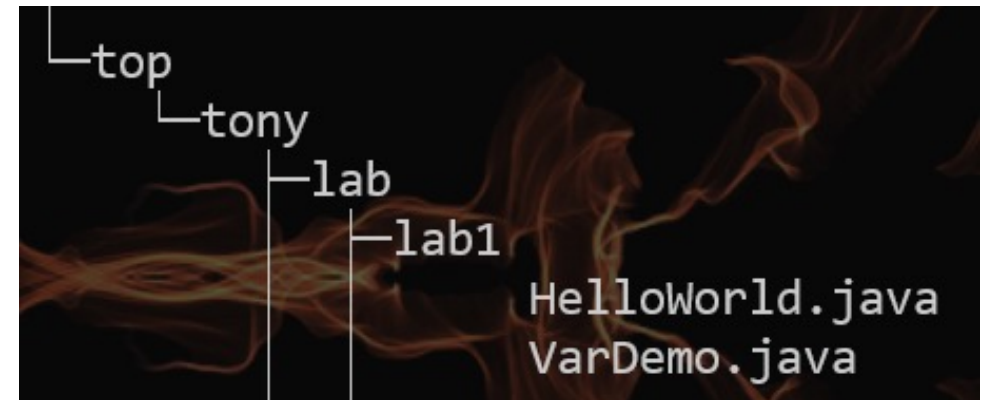
包管理的基础

- 包结构必须与目录结构相同
- 包结构下的编译与运行

```
javac .\top\tony\lab\lab1\VarDemo.java
java top.tony.lab.lab1.VarDemo
```



```
VarDemo.java x
1 package top.tony.lab.lab1;
2
3 public class VarDemo {
4     public static void main(S
5         byte b = 0x55;
6         boolean B = true;
7         short s = 0x55ff;
```



包的访问权限

如何声明包可见的类和成员？

```
default class Java  
{  
    default int a;  
}
```



只能被**同一包中**的类发现。

```
class Java  
{  
    int a;  
}
```



包之间的关系

这三个包之间有什么关系吗？继承？

```
package cn.edu.buaa;  
package cn.edu.buaa.soft;  
package cn.edu.buaa.scse;
```

包之间的关系

这三个包之间有什么关系吗？继承？

```
package cn.edu.buaa;  
package cn.edu.buaa.soft;  
package cn.edu.buaa.scse;
```



和这三个包间的关系相同。

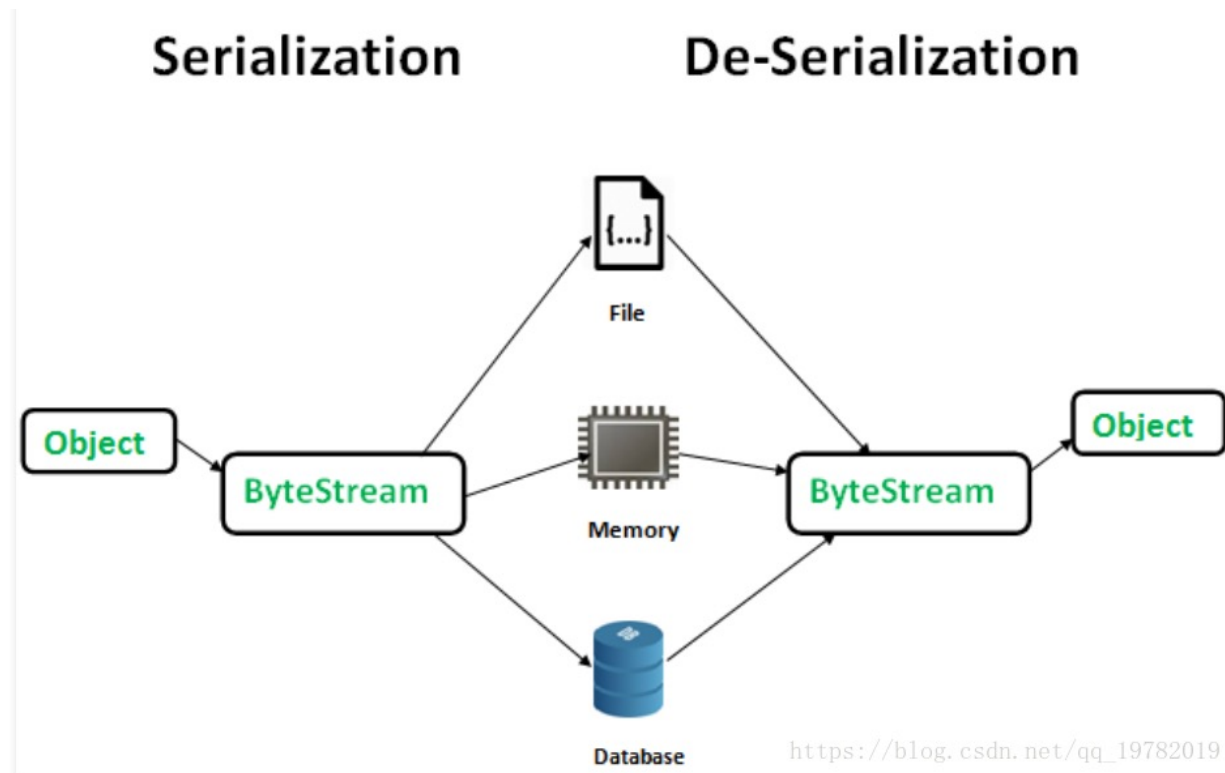
```
package a;  
package b;  
package c;
```


Serialization

Serialization in Java

Serialization

- 什么是“序列化”
- 什么是“反序列化”?
- 为什么序列化?
- 序列化结果一定是二进制吗?



https://blog.csdn.net/qq_19782019/article/details/80422143

Serialization

- 一个类是能序列化的条件?
- 哪些成员会被序列化?

序列化“实例对象”

```
// Serializable.java  
package java.io;  
public interface Serializable {  
}
```

```
// Thing.java  
class Thing implements Serializable  
{  
    private static int f1;  
    public int f2;  
    private transient int f3;  
  
    public int getValue() {  
        return f1 + f2 + f3;  
    }  
}
```

Network Programming

Network programming in Java

网络编程

- C/S, B/S
- URI, URL, TCP, UDP
- Socket
- ...

了解基本概念即可，不会考太难。

Threading

Multi-Threading in Java

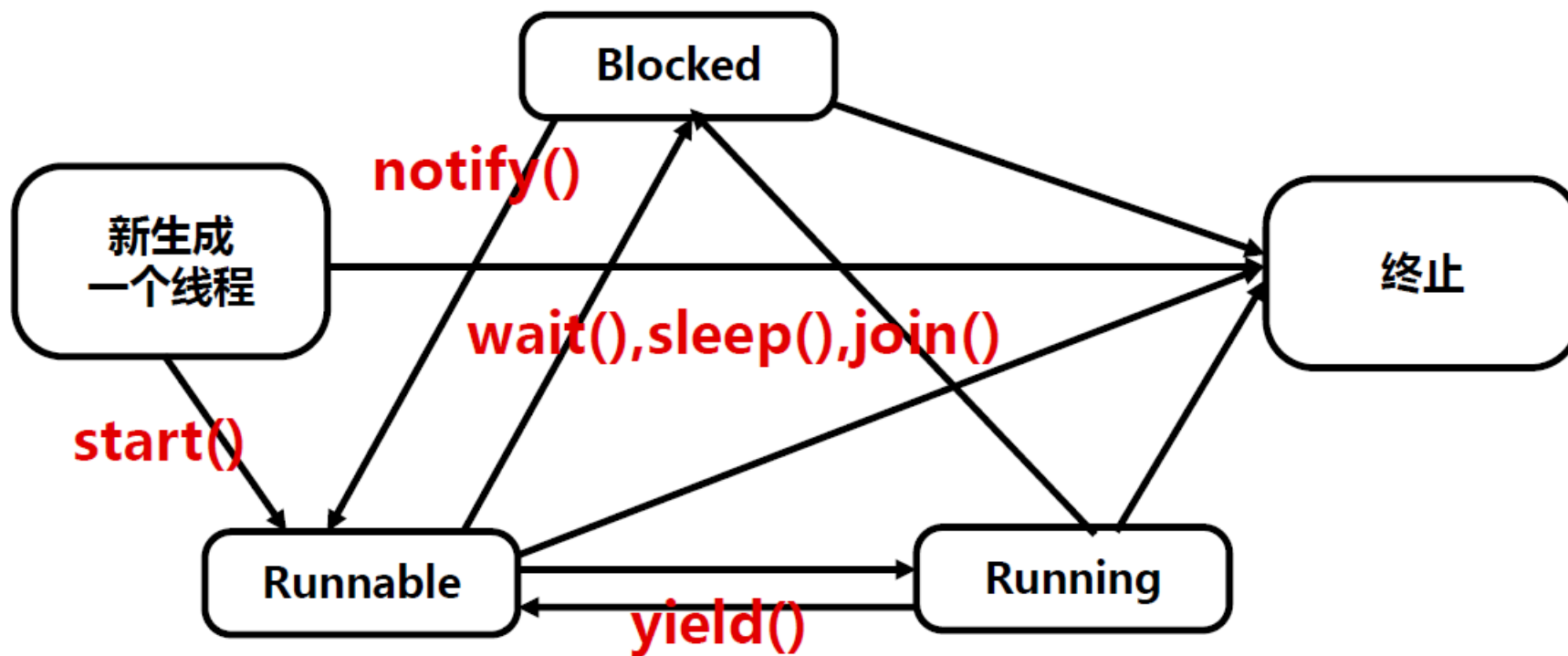
并发程序设计

- 进程？线程？协程？？
- 进程调度？线程调度？
- 线程的状态？
- 线程的生命周期？
- 线程的同步互斥？

这节 PPT 比较完善，大家熟悉相关知识即可。
不太会考编程实现。

感觉，不如 C# 的 `async await`。

线程的状态



Class+

Advanced use of class in Java

More Classes

Inner Class

```
class Java {  
    private class PrivateInner {  
    }  
  
    class DefaultInner {  
    }  
  
    static class StaticInner {  
    }  
}
```

Anonymous Class

```
interface IExecutable  
{  
    void execute();  
}  
  
var ex = new IExecutable() {  
    @Override  
    void execute() {  
        // ...  
    }  
}
```

More Classes

Inner Class

- 简单，短小
- 需要获取外部类信息
- 实现技巧

迭代器、构造器……

Anonymous Class

- 简单，短小
- 与原类/接口区别小
- **捕获外部变量**

函数式编程、偷懒……

内部类

实例内部类

- 必须由外部类实例创建

```
var outer = new OutClass();  
var inner = outer.new InnerClass();
```

- 可以访问外部类**任何**成员
- 通过 `OutClass.this` 获取外部类实例

静态内部类

- 创建与普通类相同
- 只能访问外部类静态成员
- 无法获得外部类实例

内部类与外部类不同，可以有 `protected`, `private`, `static` 修饰符

Functional Programming

Functional Programming in Java

Functional Programming

Functional programming is a style of writing computer programs that treat computations as evaluating mathematical functions.

表达式计算、条件求值、LINQ

FP Example

```
@FunctionalInterface
interface ICalc {
    int calc(int a, int b);
}

class Calculator {
    public int calc(int a, int b, ICalc calc) {
        return calc.calc(a, b);
    }
}
```

$$y = f(x)$$

FP Example

```
public class FunctionalTest {  
    public static void main(String[] args) {  
        Calculator calculator = new Calculator();  
        int magic = 66;  
  
        calculator.calc(1, 2, new ICalc() {  
            @Override  
            public int calc(int a, int b) {  
                return magic - a - b;  
            }  
        });  
        calculator.calc(1, 2, (a, b) -> magic - a - b);  
    }  
}
```

对于函数式接口，
匿名类、λ 表达式等价。

FP Summary

- Java 并不支持纯粹的函数式编程， λ 表达式与函数式接口等价。
(只作为理解，不保证是真实的 Java 底层实现。)
- 函数式接口**应满足**只有一个虚方法，只有满足该条件才能在对应该调用位置使用 λ 表达式。

Design Patterns

Where the fun begins. :)

软件设计原则

设计原则**指导**设计模式

- 单一职责原则（单一职责）
- 开闭原则（对扩展开放，对修改关闭）
- 合成复用原则（组合 instead of 继承）
- **依赖倒转原则**（面向抽象编程 instead of 面向实现编程）
- 接口隔离原则（多个专用接口 instead of 单一接口）
- 迪米特法则（减少类间的相互交流）
- **里氏替换原则**（基类可替换为子类）
- ~~你的原则~~

Inverse of Control !

设计模式

—— 设计模式 (Design Pattern) 是针对某一类问题的**最佳**解决方案，而且已经被成功应用于许多系统的设计中，它解决了在某种**特定情景中重复发生的某个问题**，即一个设计模式是从许多优秀的软件系统中总结出的成功的**可复用的设计方案**。

GoF Pattern?

设计题的加分点!

创建型模式

- 简单工厂
- 抽象工厂
- 生成器
- 工厂方法
- 原型
- 单例

抽象对象的**实例化**过程，帮助系统创建、组合和表示相关对象。

创建**什么**、**谁**创建、**怎样**创建、**何时**创建？

原型模式 (Prototype)

- 创建对象的**拷贝**
- Java 的 **Cloneable** 接口

```
// Serializable.java  
package java.lang;  
public interface Cloneable {  
}
```

```
// CloneTrooper.java  
class CloneTrooper implements Cloneable {  
    @Override  
    public Object clone() throws CloneNotSupportedException {  
        return super.clone();  
    }  
}
```

结构型模式

- **适配器**
- 桥接
- 组合
- **装饰**
- **外观**
- 享元
- 代理

解决如何**组合**类和对象（不是接口）以获得更大的结构。

行为型模式

- 职责链
- 命令
- 解释器
- 迭代器
- 中介
- 备忘录
- 观察者
- 状态
- 策略
- 模板方法
- 访问者

设计算法和对象间职责的分配，
描述它们之间的通信模式。

集中体现了**依赖倒转**的思想。

迭代器模式 (Iterator)

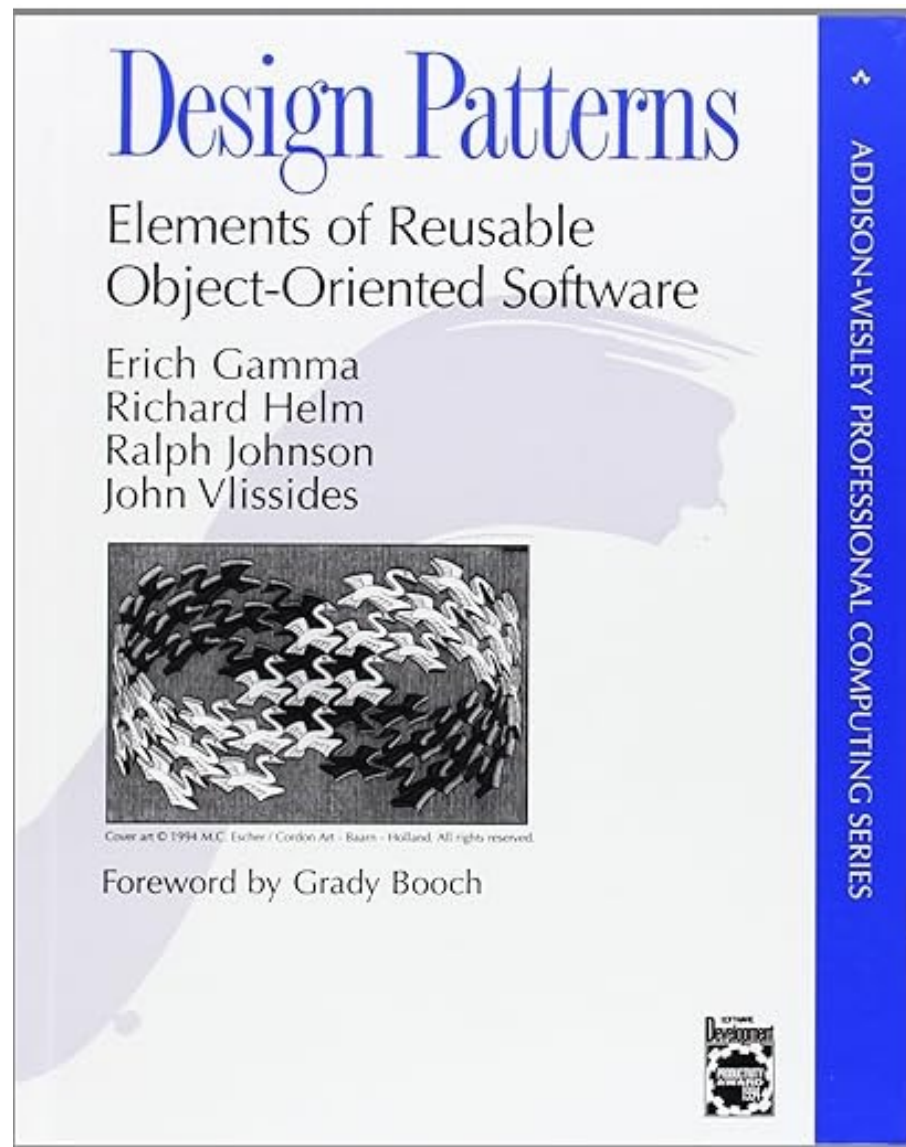
- 提供对容器的遍历操作

```
public interface Iterator<E> {  
    boolean hasNext();  
    E next();  
}
```

```
public class Sequence<E> {  
    public Iterator<E> iterator() {  
        return new SequenceIterator();  
    }  
  
    private class SequenceIterator implements Iterator<E> {  
        // ...  
    }  
}
```

推荐读物

虽然书中是 C++ 实现，但是思想是一样的。



Example: Making a Krabby Patty

- Bun
- Patty
- Topping
 - Cheese, lettuce, pickles, tomatoes, onions
- Sauce
 - Ketchup, mustard, mayonnaise, tartar sauce



Example: Making a Krabby Patty



- Topping 可**任选** (可不选)
- Mr. Krabs 只允许顾客搭配固定的 Sauce 组合。

How to make a Krabby Patty?

// Topping.java

```
public abstract class Topping {  
}
```

// Tomato.java

```
public class Tomato extends Topping {  
}
```

// Sauce.java

```
public abstract class Sauce {  
}
```

// TartarSauce.java

```
public class TartarSauce extends Sauce {  
}
```

// Patty.java

```
public class Patty {  
}
```

```

// KrabbyPatty.java
public class KrabbyPatty {
    private final Patty patty;
    private final ArrayList<Topping> toppings;
    private final ArrayList<Sauce> sauces;

    private KrabbyPatty(Patty patty, ArrayList<Topping> toppings, ArrayList<Sauce> sauces) {
        // ...
    }

    public static class Builder {
        public Builder setPatty(Patty patty) {
            // ...
        }
        public Builder addTopping(Topping topping) {
            // ...
        }
        public KrabbyPatty build() {
            return new KrabbyPatty(patty, toppings, sauces);
        }
    }
}

```

```
// ISourcePack.java
public interface ISaucePack {
    void addSauce(Collection<Sauce> sauces);
}

// DefaultSourcePack.java
public class DefaultSaucePack implements ISaucePack {
    @Override
    public void addSauce(Collection<Sauce> sauces) {
        sauces.add(new Ketchup());
        sauces.add(new TartarSauce());
    }
}

// KrabbyPatty.java
public Builder addSauce(ISaucePack saucePack) {
    saucePack.addSauce(sauces);
    return this;
}
```

// KrabbyPattyTest.java

```
KrabbyPatty patty1 = new KrabbyPatty.Builder()  
    .setPatty(new Patty())  
    .addTopping(new Lettuce())  
    .addTopping(new Tomato())  
    .addSauce(new DefaultSaucePack()).build();
```

```
KrabbyPatty patty2 = new KrabbyPatty.Builder()  
    .setPatty(new Patty())  
    .addTopping(new Cheese())  
    .addTopping(new Tomato())  
    .addSauce(sauces -> {  
        sauces.add(new Mayonnaise());  
        sauces.add(new Ketchup());  
    }).build();
```


Any questions?

预祝大家期末取得好成绩！

May the Force be with you!